

Task-1

Step 1 — Read & understand:

- **"order_estimated_delivery_date"** this is the field that that Eng.Yahia told us that ai model will predict it and based on it and the most columns we will train our model to predict before this field happen **" order_delivered_customer_date"** if order will get on time or late.
- Entities are (orders, reviews, payments, products, customers, sellers, geo_locations).
- And also about relations the order with reviews is **one-to-one** and with payments is **one-to-many** and orders with customers is **many-to-one** and orders with products and sellers is **many-to-many** and customers with geolocation is **many-to-one** and sellers with geolocation is **many-to-one**.

Step 2 — Download & ingest:

```
PS D:\MLOps_tasks\task_1> docker run --name postgres-db -e POSTGRES_PASSWORD=pass123 -d -p 5432:5432 postgres
3193fbf83f8424e9cf33d28d2729f7522d99cd30777aba3eb617340ee398cc440
```

```
from sqlalchemy import create_engine, text

default_engine = create_engine('postgresql://postgres:pass123@localhost:5432/postgres')
with default_engine.connect() as conn:
    conn.execute(text("COMMIT"))
    conn.execute(text('CREATE DATABASE "olist-db"'))

engine = create_engine('postgresql://postgres:pass123@localhost:5432/olist-db')

with open('create_table.sql', 'r') as f:
    schema_sql = f.read()

with engine.connect() as conn:
    conn.execute(text(schema_sql))
    conn.commit()

print("Schema created successfully.")
```

✓ 0.1s Python

Schema created successfully.

```
import pandas as pd

pd.read_csv('E-commerce_olist_dataset/olist_geolocation_dataset.csv').to_sql('geo_location', engine, if_exists='append', index=False)
pd.read_csv('E-commerce_olist_dataset/olist_customers_dataset.csv').to_sql('customers', engine, if_exists='append', index=False)
pd.read_csv('E-commerce_olist_dataset/olist_sellers_dataset.csv').to_sql('sellers', engine, if_exists='append', index=False)
pd.read_csv('E-commerce_olist_dataset/olist_products_dataset.csv').to_sql('products', engine, if_exists='append', index=False)
pd.read_csv('E-commerce_olist_dataset/olist_orders_dataset.csv').to_sql('orders', engine, if_exists='append', index=False)
pd.read_csv('E-commerce_olist_dataset/olist_order_payments_dataset.csv').to_sql('order_payments', engine, if_exists='append', index=False)
pd.read_csv('E-commerce_olist_dataset/olist_order_reviews_dataset.csv').to_sql('order_reviews', engine, if_exists='append', index=False)
pd.read_csv('E-commerce_olist_dataset/olist_order_items_dataset.csv').to_sql('order_items', engine, if_exists='append', index=False)
pd.read_csv('E-commerce_olist_dataset/product_category_name_translation.csv').to_sql('product_translation', engine, if_exists='append', index=False)
```

✓ 1m 54.7s Python

Step 3 — Test it:

```

query = """
SELECT o.order_id, c.customer_city, c.customer_state, p.product_category, oi.price
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
LIMIT 10;
"""

result = pd.read_sql(query, engine)
print(result)

```

Python

	order_id	customer_city	customer_state	\
0	00010242fe8c5a6d1ba2dd792cb16214	campos dos goytacazes	RJ	
1	00018f77f2f0320c557190d7a144bdd3	santa fe do sul	SP	
2	000229ec398224ef6ca0657da4fc703e	para de minas	MG	
3	00024acbcd0a6daa1e931b038114c75	atibaia	SP	
4	00042b26cf59d7ce69dfabb4e55b4fd9	varzea paulista	SP	
5	00048cc3ae777c65dbb7d2a0634bc1ea	uberaba	MG	
6	00054e8431b9d7675808bcb819fb4a32	guararapes	SP	
7	000576fe39319847cbb9d288c5617fa6	praia grande	SP	
8	0005a1a1728c9d785b8e2b08b904576c	santos	SP	
9	0005f50442cb953dcd1d21e1fb923495	jandira	SP	

	product_category	price
0	cool_stuff	58.90
1	pet_shop	239.90
2	moveis_decoracao	199.00
3	perfumaria	12.99
4	ferramentas_jardim	199.90
5	utilidades_domesticas	21.90
6	telefonica	19.90
7	ferramentas_jardim	810.00